FSD-2 Lab – Execution Commands Quick Reference

Windows CMD. Base path: D:\dev\3_1_labs\FSD-2. FIRST TIME ONLY = skip if node_modules exists (or tsc -v works). RUN = exam commands. Stop a running server with Ctrl+C before starting another (same port).

Experiment 1 – ExpressJS: Routing, HTTP Methods, Middleware

1(a). Routes, route & query parameters, URL building

Folder:

```
Exp-1/
├── index.js  <--
├── app_b.js
├── app_c.js
├── express.js
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-1
npm init -y
npm install express
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-1
node index.js
```

Expected output:

```
Server is running on http://localhost:3000
```

Check output:

1. Open http://localhost:3000/ -> Welcome to the Express.js Example!
2. Open http://localhost:3000/user/42 -> User ID is: 42
3. Open http://localhost:3000/search?query=laptop -> Searching for: laptop
4. Open http://localhost:3000/product/shoes?page=3 -> ...Viewing page 3. URL: /product/shoes?page=3

1(b). HTTP methods: accept, retrieve, delete

Folder:

```
Exp-1/
├── index.js
├── app_b.js  <--
├── app_c.js
├── express.js
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-1
npm init -y
npm install express
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-1
node app_b.js
```

Expected output:

```
Server is running on http://localhost:3000
```

Check output:

1. Browser: GET http://localhost:3000/student -> student data retrieved Successfully
2. Postman: POST http://localhost:3000/add -> student data added successfully
3. Postman: DELETE http://localhost:3000/student/101 -> student with ID 101 deleted
4. Postman: POST http://localhost:3000/student, Body > raw > JSON:
{"name":"Rahul","email":"rahul@example.com","message":"Hello"} -> Form submitted page (JSON only; a form body fails)
5. POST and DELETE cannot be tested from the browser address bar.

1(c). Middleware

Folder:

```
Exp-1/
├── index.js
├── app_b.js
├── app_c.js  <--
├── express.js
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-1
npm init -y
npm install express
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-1
node app_c.js
```

Expected output:

```
Server is running on http://localhost:3000
```

Check output:

1. Open http://localhost:3000/ -> Hello, world!
2. Open http://localhost:3000/about -> About page-Middleware is working!
3. Terminal prints: GET request made to: / and GET request made to: /about

Experiment 2 – ExpressJS: Templating, Form Data

2(a). Templating engine (EJS)

Folder:

```
Exp-2/
├── app.js  <--
├── app_b.js
├── views/
│   ├── index.ejs  <--
│   └── about.ejs  <--
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-2
npm init -y
npm install express ejs
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-2
node app.js
```

Expected output:

```
Server running at http://localhost:3000
```

Check output:

1. Open http://localhost:3000/ -> heading: welcome to express with EJS templations!
2. Open http://localhost:3000/about -> heading: This is the About page of the app, with Back to homepage link

2(b). Form data

Folder:

```
Exp-2/
├── app.js
├── app_b.js  <--
├── views/
│   ├── index.ejs
│   └── about.ejs
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-2
npm init -y
npm install express
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-2
node app_b.js
```

Expected output:

```
Server is running on http://localhost:3000
```

Check output:

1. Open http://localhost:3000/ -> Contact Form
2. Enter Name: Rahul, Email: rahul@example.com, Message: Hello there. Click Submit.
3. Verify the Form submitted page shows the three values; click Go Back.

Experiment 3 – ExpressJS: Cookies, Sessions, Authentication

3(a). Session management (cookies and sessions)

Folder:

```
Exp-3/
├── app_a.js  <--
├── app_b.js
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-3
npm init -y
npm install express express-session
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-3
node app_a.js
```

Expected output:

```
Server is running on http://localhost:3000
```

Check output:

1. Open http://localhost:3000/ -> Home Page, click Login.
2. Login with username: user, password: password -> Welcome back, user!
3. F12 > Application > Cookies > localhost:3000 -> cookie connect.sid.
4. Click Logout -> back to Home Page.

3(b). User authentication

Folder:

```
Exp-3/
├── app_a.js
├── app_b.js  <--
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-3
npm init -y
npm install express express-session bcryptjs
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-3
node app_b.js
```

Expected output:

```
Server is running on http://localhost:3000
```

Check output:

1. Open http://localhost:3000/ -> redirects to /login.
2. Open http://localhost:3000/register, register siva / pass123 -> Registration successful!
3. Open /login, login siva / pass123 -> Welcome siva. Wrong password -> Invalid username or password.
4. Click Logout. (Users are lost when the server restarts.)

Experiment 4 – ExpressJS: Database, RESTful APIs

4(a). MongoDB with Mongoose: CRUD

Folder:

```
Exp-4/
├── app_a.js  <--
├── server.js
├── public/
│   └── index.html
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-4
npm init -y
npm install express mongoose cors
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-4
node app_a.js
```

Expected output:

```
MongoDB connected successfully
CREATE:  { name: 'Rahul', age: 20, ... }
READ:    [ ... ]
UPDATE:  { ... age: 21 ... }
DELETE:  { ... }
```

Check output:

1. Needs internet and MongoDB Atlas access (connection string is inside the file).
2. Check the terminal shows CREATE, READ, UPDATE (age 21) and DELETE. The program ends by itself.

4(b). Single-page application using REST APIs

Folder:

```
Exp-4/
├── app_a.js
├── server.js  <--
├── public/
│   └── index.html  <--
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-4
npm init -y
npm install express mongoose cors
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-4
npm start
```

Expected output:

```
Server running at http://localhost:3000
MongoDB connected
```

Check output:

1. Open http://localhost:3000/ -> Student Management SPA.
2. Enter Name, Age, Department, Email and click Add Student -> new row appears. Click Delete -> row removed.
3. Browser: GET http://localhost:3000/api/students -> JSON list.
4. Postman: PUT http://localhost:3000/api/students/<_id>, raw JSON {"age":22} -> updated record.

Experiment 5 – ReactJS (Vite): Render HTML, JSX, Components

5(a). Render HTML to a webpage

Folder:

```
Exp-5/
├── package.json
├── main_b.txt
├── main_c.txt
└── src/
    ├── main.jsx  <--
    └── App.jsx  <--
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> Welcome to ReactJS / This is a simple React application.
2. src\main.jsx must contain the lines below (this is the saved 5(a) version).

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <App />
);
```

5(b). JSX markup

Folder:

```
Exp-5/
├── package.json
├── main_b.txt  <--
├── main_c.txt
└── src/
    ├── main.jsx  <--
    └── App.jsx
```

Commands:

FIX FIRST (one time)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
copy /Y main_b.txt src\main.jsx
```

main_b.txt has typos. In src\main.jsx change:

```
ORIGINAL   : import React form "react"
REPLACEMENT: import React from "react";

ORIGINAL   : const toot=ReactDOM.createRoot(document.getElementById("root");
REPLACEMENT: const root=ReactDOM.createRoot(document.getElementById("root"));
```

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> Welcome to React, This is written using JSX, and a 3-item list.
2. Afterwards restore src\main.jsx as shown under 5(a).

5(c). Function and class components (nested)

Folder:

```
Exp-5/
├── package.json
├── main_b.txt
├── main_c.txt  <--
└── src/
    ├── main.jsx  <--
    └── App.jsx
```

Commands:

FIX FIRST (one time)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
copy /Y main_c.txt src\main.jsx
```

main_c.txt has errors. In src\main.jsx change:

```
ORIGINAL   : import React,{component} from "react";
REPLACEMENT: import React,{ Component } from "react";

ORIGINAL   : import ReactDOM from "react.dom/client";
REPLACEMENT: import ReactDOM from "react-dom/client";

ORIGINAL   : return <h1>Reac components<h1>;
REPLACEMENT: return <h1>React components</h1>;

ORIGINAL   : <content/>
REPLACEMENT: <Content/>
```

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-5
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> heading Header (function component) + paragraph This is a class component.
2. Afterwards restore src\main.jsx as shown under 5(a).

Experiment 6 – ReactJS (Vite): Props, States, Styles, Events

6(a). Props and state

Folder:

```
Exp-6/
├── App_b.css
├── package.json
└── src/
    ├── main.jsx  <--
    ├── App_a.jsx  <--
    ├── App_b.jsx
    └── App_c.jsx
```

Commands:

SWITCH COMPONENT

```
src\main.jsx   (line 4)
Currently : import App from './App_c.jsx'
Change to : import App from './App_a.jsx'
```

Change the import filename, save, and check the browser. Keep the Vite server running.

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-6
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-6
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> Student Name: Rahul, Count: 0.
2. Click Increment -> Count increases.

6(b). CSS styling and displaying data

Folder:

```
Exp-6/
├── App_b.css  <--
├── package.json
└── src/
    ├── main.jsx  <--
    ├── App_a.jsx
    ├── App_b.jsx  <--
    └── App_c.jsx
```

Commands:

FIX FIRST (one time)

```
src\main.jsx   (line 4)
Currently : import App from './App_c.jsx'
Change to : import App from './App_b.jsx'
```

Change the import filename, save, and check the browser. Keep the Vite server running.

App_b.jsx is broken (no function, wrong variable, CSS outside src). Fix:

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-6
copy /Y App_b.css src\App_b.css
```

```
File: src\App_b.jsx

ADD just before  "return (" :
function App() {
  const student = students[0];

ADD just after the closing  ");"  line (before export default App;) :
}
```

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-6
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-6
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> blue card: Student Details, Name: Rahul, Course: B.Tech, Marks: 85%.
2. Sass: CODE NOT FOUND (only plain CSS exists).

6(c). Respond to events

Folder:

```
Exp-6/
├── App_b.css
├── package.json
└── src/
    ├── main.jsx  <--
    ├── App_a.jsx
    ├── App_b.jsx
    └── App_c.jsx  <--
```

Commands:

SWITCH COMPONENT

```
src\main.jsx   (line 4)
Saved default: import App from './App_c.jsx'
```

If you changed it for 6(a)/6(b), set it back to App_c.jsx. Change the import filename, save, and check the browser. Keep the Vite server running.

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-6
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-6
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> Click the button.
2. Click Click Me -> Button clicked successfully.

Experiment 7 – ReactJS (Vite): Conditional Rendering, Lists, Forms

7(a). Conditional rendering

Folder:

```
Exp-7/
├── package.json
└── src/
    ├── main.jsx  <--
    ├── App_a.jsx  <--
    ├── App_b.jsx
    └── App_c.jsx
```

Commands:

SWITCH COMPONENT

```
src\main.jsx   (line 4)
Currently : import App from './App_c.jsx'
Change to : import App from './App_a.jsx'
```

Change the import filename, save, and check the browser. Keep the Vite server running.

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-7
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-7
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> Please Login.
2. Click Login -> Welcome User! (button becomes Logout). Click again to go back.

7(b). Rendering lists

Folder:

```
Exp-7/
├── package.json
└── src/
    ├── main.jsx  <--
    ├── App_a.jsx
    ├── App_b.jsx  <--
    └── App_c.jsx
```

Commands:

SWITCH COMPONENT

```
src\main.jsx   (line 4)
Currently : import App from './App_c.jsx'
Change to : import App from './App_b.jsx'
```

Change the import filename, save, and check the browser. Keep the Vite server running.

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-7
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-7
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> Student List: Rahul, Amit, Sneha, Karan.

7(c). Different form fields

Folder:

```
Exp-7/
├── package.json
└── src/
    ├── main.jsx  <--
    ├── App_a.jsx
    ├── App_b.jsx
    └── App_c.jsx  <--
```

Commands:

FIX FIRST (one time)

Saved default import is App_c.jsx. App_c.jsx is incomplete (no gender, course or Submit button). In src\App_c.jsx replace this line:

```
ORIGINAL   : {/* Remaining fields (gender, course, checkbox, submit) as per notes */}
REPLACEMENT: the lines below
```

```
<label>Gender:</label>
<input type="radio" name="gender" value="Male"
  onChange={(e) => setGender(e.target.value)} /> Male
<input type="radio" name="gender" value="Female"
  onChange={(e) => setGender(e.target.value)} /> Female<br /><br />
<label>Course:</label>
<select value={course} onChange={(e) => setCourse(e.target.value)}>
  <option value="">-- Select --</option>
  <option value="B.Tech">B.Tech</option>
  <option value="M.Tech">M.Tech</option>
  <option value="MCA">MCA</option>
</select><br /><br />
<button type="submit">Submit</button>
```

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-7
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-7
npm run dev
```

Expected output:

```
VITE v8.x  ready in ___ ms
  Local:   http://localhost:5173/
```

Check output:

1. Open http://localhost:5173/ -> Student Registration Form.
2. Enter Name, Email, choose Gender and Course, click Submit.
3. Verify the alert shows Name, Email, Gender and Course.

Experiment 8 – ReactJS: React Router, Updating the Screen

8(a). Routing with React Router (Create React App)

Folder:

```
exp-9a/
├── package.json
└── src/
    ├── index.js
    ├── App.js  <--
    ├── Home.js  <--
    └── About.js  <--
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\exp-9a
npm install
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\exp-9a
npm start
```

Expected output:

```
Compiled successfully!
  Local:  http://localhost:3000
```

Check output:

1. Run from exp-9a (the Exp-8 folder has no entry file or start script).
2. Browser opens http://localhost:3000/ -> Home | About links, Home Page.
3. Click About -> About Page, URL becomes /about. Click Home to return.

8(b). Updating the screen

CODE NOT FOUND

Experiment 9 – ReactJS: Hooks, Sharing Data Between Components

9(a). Hooks

CODE NOT FOUND

Experiment 11 – Node.js

11(a). Web server with Node.js

Folder:

```
Exp-11/
├── server_a.js  <--
├── server_b.js
├── createfile.js
├── src.txt
├── urlParser.js
├── mathModule.js
└── app.js
```

Commands:

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-11
node server_a.js
```

Expected output:

```
Server is running at http://localhost:3000
```

Check output:

1. No installation needed (built-in http module).
2. Open http://localhost:3000 -> Welcome to Node.js Web Server! with current date and time. Refresh to see the time change.

11(b). Transfer data over HTTP

Folder:

```
Exp-11/
├── server_a.js
├── server_b.js  <--
├── createfile.js
├── src.txt
├── urlParser.js
├── mathModule.js
└── app.js
```

Commands:

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-11
node server_b.js
```

Expected output:

```
Server is running at http://localhost:3000
```

Check output:

1. Press Ctrl+C on 11(a) first.
2. Open http://localhost:3000 -> Hello! Data is transferred successfully using HTTP protocol.

11(c). Create src.txt (fs module)

Folder:

```
Exp-11/
├── server_a.js
├── server_b.js
├── createfile.js  <--
├── src.txt  <--
├── urlParser.js
├── mathModule.js
└── app.js
```

Commands:

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-11
del src.txt
node createfile.js
type src.txt
```

Expected output:

```
src.txt created successfully.
HTML
CSS
Javascript
Typescript
MongoDB
Express.js
React.js
Node.js
```

Check output:

1. Check the terminal message.
2. Verify src.txt lists the 8 technologies (type src.txt, or open it).

11(d). Parse a URL (url module)

Folder:

```
Exp-11/
├── server_a.js
├── server_b.js
├── createfile.js
├── src.txt
├── urlParser.js  <--
├── mathModule.js
└── app.js
```

Commands:

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-11
node urlParser.js
```

Expected output:

```
Complete URL : https://www.example.com:8080/products/item?id=101&name=Laptop#details
Protocol     : https:
Hostname     : www.example.com
Port         : 8080
Pathname     : /products/item
Query String : ?id=101&name=Laptop
ID           : 101
Name         : Laptop
Hash         : #details
```

Check output:

1. Check the terminal output.

11(e). User-defined module

Folder:

```
Exp-11/
├── server_a.js
├── server_b.js
├── createfile.js
├── src.txt
├── urlParser.js
├── mathModule.js  <--
└── app.js  <--
```

Commands:

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-11
node app.js
```

Expected output:

```
Addition: 30
Subtraction: 10
```

Check output:

1. Check the terminal output. (mathModule.js is the module; run app.js.)

Experiment 12 – TypeScript

No package.json or tsconfig.json in Exp-12: uses global tsc. Compile creates the .js file in the same folder.

12(a). Simple and special types

Folder:

```
Exp-12/
├── types.ts  <--
├── function.ts
├── arrowParams.ts
├── classDemo.ts
├── nameSpace.ts
├── mathModule.ts
├── main.ts
└── generics.ts
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
npm install -g typescript
tsc -v
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-12
tsc types.ts
node types.js
```

Expected output:

```
---- Simple Types ----
Number: 20
String: Lalitha
Boolean: true
Null (Height): null
Any: Can be anything
Void: undefined

---- Special Types ----
Array: [ 85, 90, 95 ]
Tuple: [ 101, 'Raji', true ]
Enum (Tuesday): 2
Union (Value): TypeScript
```

Check output:

1. tsc may print error TS2451 about 'name'; types.js is still created. Ignore it and run node types.js.
2. Check the terminal output.

12(b). Function parameter and return types

Folder:

```
Exp-12/
├── types.ts
├── function.ts  <--
├── arrowParams.ts
├── classDemo.ts
├── nameSpace.ts
├── mathModule.ts
├── main.ts
└── generics.ts
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
npm install -g typescript
tsc -v
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-12
tsc function.ts
node function.js
```

Expected output:

```
Addition: 30
Subtraction: 35
Multiplication: 42
Hello, TypeScript!
```

Check output:

1. Check the terminal output.

12(c). Arrow functions: optional, default, rest

Folder:

```
Exp-12/
├── types.ts
├── function.ts
├── arrowParams.ts  <--
├── classDemo.ts
├── nameSpace.ts
├── mathModule.ts
├── main.ts
└── generics.ts
```

Commands:

FIX FIRST (one time)

arrowParams.ts wrongly contains the Person class (same as classDemo.ts). Replace the whole file with:

```
const greet = (name: string, age?: number): string => {
    if (age !== undefined) {
        return `Hello ${name}, you are ${age} years old.`;
    }
    return `Hello ${name}!`;
};

const welcome = (name: string, city: string = "Hyderabad"): string => {
    return `Welcome ${name} from ${city}`;
};

const sum = (...numbers: number[]): number => {
    return numbers.reduce((acc, curr) => acc + curr, 0);
};

console.log(greet("Ravi"));
console.log(greet("Ravi", 25));
console.log(welcome("Sneha"));
console.log(welcome("Sneha", "Bangalore"));
console.log("Sum:", sum(10, 20, 30, 40));
```

FIRST TIME ONLY (skip if already installed)

```
npm install -g typescript
tsc -v
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-12
tsc arrowParams.ts
node arrowParams.js
```

Expected output:

```
Hello Ravi!
Hello Ravi, you are 25 years old.
Welcome Sneha from Hyderabad
Welcome Sneha from Bangalore
Sum: 100
```

Check output:

1. Check the terminal output.

12(d). Classes, constructors, access specifiers

Folder:

```
Exp-12/
├── types.ts
├── function.ts
├── arrowParams.ts
├── classDemo.ts  <--
├── nameSpace.ts
├── mathModule.ts
├── main.ts
└── generics.ts
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
npm install -g typescript
tsc -v
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-12
tsc classDemo.ts
node classDemo.js
```

Expected output:

```
Name: Alice
Name: Alice, Age: 22, City: Hyderabad, ID: 101
Age (via method): 22
```

Check output:

1. Check the terminal output.

12(e). Namespaces and modules

Folder:

```
Exp-12/
├── types.ts
├── function.ts
├── arrowParams.ts
├── classDemo.ts
├── nameSpace.ts  <--
├── mathModule.ts  <--
├── main.ts  <--
└── generics.ts
```

Commands:

FIX FIRST (one time)

Modules only: plain tsc gives ES-module output that node cannot run, so use --module commonjs. Also type the parameters in mathModule.ts:

```
ORIGINAL   : export function multiply(a, b) {
REPLACEMENT: export function multiply(a: number, b: number): number {

ORIGINAL   : export function divide(a, b) {
REPLACEMENT: export function divide(a: number, b: number): number {
```

FIRST TIME ONLY (skip if already installed)

```
npm install -g typescript
tsc -v
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-12
tsc nameSpace.ts
node nameSpace.js

tsc main.ts --module commonjs
node main.js
```

Expected output:

```
Namespace Add: 30
Namespace Subtract: 10

Module Multiply: 42
Module Divide: 10
```

Check output:

1. Check the terminal output for both programs.
2. Modules: main.ts imports mathModule.ts; tsc compiles both.

12(f). Generics

Folder:

```
Exp-12/
├── types.ts
├── function.ts
├── arrowParams.ts
├── classDemo.ts
├── nameSpace.ts
├── mathModule.ts
├── main.ts
└── generics.ts  <--
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
npm install -g typescript
tsc -v
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-12
tsc generics.ts
node generics.js
```

Expected output:

```
Identity (number): 100
Identity (string): Hello TypeScript
Pair: [ 'Age', 25 ]
Length of string: 10
Length of array: 4
```

Check output:

1. Check the terminal output.

Experiment 15 – Augmented Programs

15(a). CSS 2D and 3D transformations

CODE NOT FOUND

15(b). HTML5 and CSS3 webpage

CODE NOT FOUND

15(c). JavaScript to-do list

CODE NOT FOUND

15(d). Node.js and Express.js to-do list

Folder:

```
Exp-15/
├── app.js  <--
├── index.html  <--
└── package.json
```

Commands:

FIRST TIME ONLY (skip if already installed)

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-15
npm init -y
npm install express
```

RUN

```
cd /d D:\dev\3_1_labs\FSD-2\Exp-15
node app.js
```

Expected output:

```
Server is running at http://localhost:3000
```

Check output:

1. Keep the server running. Double-click Exp-15\index.html to open it in the browser.
2. Type a task (Buy milk), click Add Task -> alert Task Added Successfully; click Show Tasks to list.
3. Open http://localhost:3000/list -> ["Buy milk"]. Tasks are lost when the server restarts.

15(e). React.js quiz application

CODE NOT FOUND

15(f). MongoDB University certification

CODE NOT FOUND